G — Serving efficiency and retrieval grounding

0. Role in the system

This section informs the **serving layer** and the **Library**. Serving: a dense ~9B worker held entirely in 12 GB of VRAM; a ~35B-total / ~3B-active (35B-A3B) mixture-of-experts senior whose expert tensors live in 32 GB of system RAM and are computed by the CPU under llama.cpp; speculative decoding of the worker with a same-family draft model; 4-bit weight-only quantization of both models (and re-quantization after each weekly LoRA merge); KV-cache sizing, quantization and reuse; FlashAttention enabled for prefill. Library: retrieval over the developer's own docs and code with hybrid lexical+dense first-stage search, late-interaction (ColBERT MaxSim) or cross-encoder reranking, and a **hard context budget** with position-aware packing, because small models lose information placed in the middle of long contexts.

After reading this pack an agent must be able to: (1) implement or configure the speculative-decoding step and pick the draft length from measured acceptance rate; (2) compute weight, KV-cache and bandwidth budgets for any model shape and choose quantization levels that fit 12 GB VRAM / 32 GB RAM; (3) implement GPTQ- or AWQ-style calibration-based quantization (or drive llama.cpp's equivalent) with the right calibration set; (4) build the Library index (BM25 + dense + ColBERT token embeddings), the two-stage retrieve-then-rerank pipeline, and the budgeted, position-aware context packer; (5) run the evaluations that show each piece works.

1. Papers

1.1 Fast Inference from Transformers via Speculative Decoding (Leviathan et al., 2023) — arXiv:2211.17192

Problem and claim. Autoregressive decoding from a large target model M_p produces one token per serial forward pass, and at small batch the pass is bound by reading the weights, not by arithmetic. The paper's claim: let a cheap approximation model M_q draft γ tokens, verify all of them with one parallel pass of M_p, and accept/reject with a rule that makes the output distribution **exactly** that of M_p. No retraining, no architecture change, works at any temperature. Reported 2X–3X wall-time speedup on T5-XXL with identical outputs.

Headline results. Table 2, T5-XXL (11B) target, batch size 1, single TPU-v4, speedup relative to T5X decoding:

Task	Draft	Temp	γ	α	Speedup
EnDe	T5-small (77M)	0	7	0.75	3.4X
EnDe	T5-base (250M)	0	7	0.80	2.8X
EnDe	T5-large (800M)	0	7	0.82	1.7X
EnDe	T5-small	1	7	0.62	2.6X
EnDe	T5-base	1	5	0.68	2.4X
EnDe	T5-large	1	3	0.71	1.4X
CNN/DM	T5-small	0	5	0.65	3.1X
CNN/DM	T5-base	0	5	0.73	3.0X
CNN/DM	T5-large	0	3	0.74	2.2X
CNN/DM	T5-small	1	5	0.53	2.3X
CNN/DM	T5-base	1	3	0.55	2.2X
CNN/DM	T5-large	1	3	0.56	1.7X

The smallest draft wins: larger drafts raise α but raise the cost coefficient c more. Greedy (temp 0) gives higher α than sampling (temp 1). Also tested: a 97M GPT-like decoder on lm1b with a 6M draft and with unigram/bigram drafts; LaMDA 137B with 100M/2B/8B drafts (numbers not transcribed here).

Method — equations.

Acceptance rule (Section 2.3; implemented per token in Algorithm 1). Draw $x \sim q(x)$. Keep it if $q(x) \leq p(x)$; otherwise reject it with probability $1 - p(x)/q(x)$. With $r_i \sim U(0,1)$, draft token i is rejected iff

$$r_i > \frac{p_i(x)}{q_i(x)}$$

i.e. it is accepted with probability $\min\left(1, \frac{p(x)}{q(x)}\right)$. Here p is the **target** (M_p) distribution and q the **draft** (M_q) distribution, both already adjusted for the sampling method (temperature, top-k). Note: the task brief wrote the test as $\min(1, q/p)$; the paper's form is $\min(1, p/q)$, and Theorem 3.5's proof uses that form.

Adjusted (residual) distribution sampled after a rejection (Section 2.3; normalized form in Appendix A.1, where β is the acceptance rate):

$$p'(x) = \text{norm}(\max(0, p(x) - q(x))) = \frac{p(x) - \min(q(x), p(x))}{1 - \beta}$$

Definition 3.1 (acceptance rate): $\beta_{x_{<t}}$ is the probability that $x_t \sim q(x_t \mid x_{<t})$ is accepted. Under the i.i.d. simplification $\alpha = E(\beta)$.

Equation (1), expected tokens per iteration (one M_p call):

$$E(\#\text{generated tokens}) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

Definition 3.2 with $M(x) = \frac{p(x)+q(x)}{2}$: $D_{LK}(p,q) = \sum_x |p(x) - M(x)| = \sum_x |q(x) - M(x)|$. Lemma 3.3: $D_{LK}(p,q) = 1 - \sum_x \min(p(x), q(x))$. Theorem 3.5: $\beta = 1 - D_{LK}(p,q)$. Corollary 3.6:

$$\alpha = 1 - E(D_{LK}(p,q)) = E(\min(p,q))$$

Definition 3.7: cost coefficient c = (time of one M_q run) / (time of one M_p run). Theorem 3.8, expected wall-time improvement factor:

$$\frac{1 - \alpha^{\gamma+1}}{(1-\alpha)(\gamma c + 1)}$$

Corollary 3.9: if $\alpha > c$ some γ gives an improvement, and $\gamma = 1$ already gives at least $\frac{1+\alpha}{1+c}$. Theorem 3.11, increase in arithmetic operations, with $\hat{c}$ the ratio of arithmetic operations per token of M_q to M_p:

$$\frac{(1-\alpha)(\gamma\hat{c} + \gamma + 1)}{1 - \alpha^{\gamma+1}}$$

Section 3.5: $\gamma^* = \arg\max_{\gamma \in \mathbb{Z}^+} \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}$; no closed form, found numerically (Figure 3).

Method — algorithm. Algorithm 1, SpeculativeDecodingStep(M_p, M_q, prefix):

```
1. for i = 1..gamma:                      # draft
       q_i <- M_q(prefix + [x_1..x_{i-1}]);  x_i ~ q_i
2. in parallel: p_1..p_{gamma+1} <- M_p(prefix), M_p(prefix+[x_1]), ..., M_p(prefix+[x_1..x_gamma])
3. r_1..r_gamma ~ U(0,1) i.i.d.
4. n <- min({ i-1 : 1<=i<=gamma, r_i > p_i(x_i)/q_i(x_i) } ∪ {gamma})
5. p' <- p_{n+1};  if n < gamma: p' <- norm(max(0, p_{n+1} - q_{n+1}))
6. t ~ p'
7. return prefix + [x_1..x_n, t]          # between 1 and gamma+1 new tokens
```

Hyperparameters and setup. $\gamma \in [2, 10]$ in the tables; $c < 0.05$ for the small drafts and about 0.11 for T5-large (Table 4); α estimated over 10K target-generated tokens; LaMDA outputs pass a top-40 filter; an optional lenience parameter $l \in [0,1]$ (Appendix A.5) trades exactness for speed and is not used in the main results.

Ablations that matter. Draft size: α 0.75 → 0.82 (small → large) but speedup 3.4X → 1.7X because c grows. Temperature: α drops roughly 0.1–0.2 from greedy to T=1. Operations overhead (Table 1): 1.11X–1.63X more arithmetic, worse at low α. An oracle choosing γ per step would add up to ~60% (estimate; left to future work).

Code / data / license. No code link in the fetched text.

Stated limitations. Needs spare parallel compute; rejected drafts waste arithmetic; the wall-time analysis assumes i.i.d. acceptance and long generations; only T5 implemented; beam search left to future work (Appendix A.4); lenience changes outputs. A positive side effect: target weights and KV cache are read once per iteration instead of once per token.

Use in our design. Run the worker with a same-family draft ~1/50–1/100 its size (so $c \lesssim 0.05$), greedy or low temperature for code (higher α). Measure α on the developer's own traffic over ≥10K tokens, then set γ by

maximizing Theorem 3.8; expect γ 3–7 for α 0.6–0.8. For the MoE senior, speculative gains shrink because a batch of $\gamma+1$ tokens touches more experts (see Section 2.2 derivation) — measure before enabling. PDF: https://arxiv.org/pdf/2211.17192

1.2 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (Dao et al., 2022) — arXiv:2205.14135

Problem and claim. Exact attention materializes the $N \times N$ score matrix in GPU HBM, so time and memory are dominated by HBM reads/writes, not FLOPs; approximate-attention methods cut FLOPs but rarely wall time. Claim: an IO-aware exact algorithm that tiles Q, K, V into blocks that fit on-chip SRAM, computes softmax incrementally (online softmax) and never writes the $N \times N$ matrix, achieves $\Theta\left(N^2 d^2 M^{-1}\right)$ HBM accesses versus $\Theta\left(Nd + N^2\right)$ and is provably optimal up to constants over SRAM sizes.

Headline results. GPT-2 medium attention (seq 1024, head dim 64, 16 heads, batch 64, A100, fwd+bwd; Figure 2 left): PyTorch 66.6 GFLOPs / 40.3 GB HBM R/W / 41.7 ms; FlashAttention 75.2 GFLOPs / 4.4 GB / 7.3 ms (more FLOPs, 9x fewer HBM bytes, 5.7x faster). Training (8xA100): BERT-large seq 512, 17.4 ± 1.4 min vs MLPerf 1.1 record 20.0 ± 1.5 min (15% faster); GPT-2 small on OpenWebText 2.7 days vs HuggingFace 9.5 (3.5x) and Megatron-LM 4.7 (2.0x); GPT-2 medium 6.9 vs 21.0 / 11.5 days. Long-Range Arena (1K–4K): 2.4x (dense) and 2.8x (block-sparse) vs baseline. Path-X (16K) 61.4% (first Transformer above chance); Path-256 (64K) 63.1% with block-sparse. Attention micro-benchmark: up to 3x faster than PyTorch for lengths 128–2K, up to 7.6x on GPT-2 attention, memory linear in N and "up to 20x more memory efficient than exact attention baselines" (length at which that peak occurs not stated in fetched text). A100 numbers used by the paper: HBM 40–80 GB at 1.5–2.0 TB/s; SRAM 192 KB per each of 108 SMs at roughly 19 TB/s.

Method — equations.

Standard attention for $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}\left(\mathbf{S}\right) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}$$

Softmax decomposition for a block $x \in \mathbb{R}^B$:

$$m\left(x\right) := \max_i x_i, \quad f\left(x\right) := \left[\, e^{x_1 - m\left(x\right)} \; \ldots \; e^{x_B - m\left(x\right)} \,\right], \quad \ell\left(x\right) := \sum_i f\left(x\right)_i, \quad \mathrm{softmax}\left(x\right) := \frac{f\left(x\right)}{\ell\left(x\right)}$$

Online-softmax merge for $x = \left[x^{(1)} \; x^{(2)}\right] \in \mathbb{R}^{2B}$:

$$m\left(x\right) = \max\left(m\left(x^{(1)}\right), m\left(x^{(2)}\right)\right), \quad f\left(x\right) = \left[e^{m\left(x^{(1)}\right) - m\left(x\right)} f\left(x^{(1)}\right) \quad e^{m\left(x^{(2)}\right) - m\left(x\right)} f\left(x^{(2)}\right)\right]$$

$$\ell\left(x\right) = e^{m\left(x^{(1)}\right) - m\left(x\right)} \ell\left(x^{(1)}\right) + e^{m\left(x^{(2)}\right) - m\left(x\right)} \ell\left(x^{(2)}\right)$$

N = sequence length, d = head dimension, M = SRAM size (elements).

Theorem 1: Algorithm 1 returns $\mathbf{O} = \mathrm{softmax}\left(\mathbf{Q}\mathbf{K}^\top\right)\mathbf{V}$ with $O\left(N^2 d\right)$ FLOPs and $O\left(N\right)$ additional memory beyond inputs and output. Theorem 2: for $d \le M \le Nd$, standard attention requires $\Theta\left(Nd + N^2\right)$ HBM accesses while FlashAttention requires $\Theta\left(N^2 d^2 M^{-1}\right)$. Proposition 3: for $d \le M \le Nd$, no algorithm computes exact attention with $o\left(N^2 d^2 M^{-1}\right)$ HBM accesses for all M in $\left[d, Nd\right]$. Proposition 4 (block-sparse, Algorithm 5): $\Theta\left(Nd + N^2 d^2 M^{-1} s\right)$ HBM accesses, s = fraction of nonzero blocks in the block-sparsity mask.

Method — algorithm. Algorithm 1 (FlashAttention forward), inputs in HBM:

```
1.  B_c = ceil(M / (4d));  B_r = min(ceil(M / (4d)), d)
2.  O = 0_{N x d};  l = 0_N;  m = (-inf)_N            (in HBM)
3.  split Q into T_r = ceil(N/B_r) blocks Q_i (B_r x d); split K, V into T_c = ceil(N/B_c) blocks K_j, V_j (B_c
x d)
4.  split O into O_i (B_r x d), l into l_i, m into m_i (length B_r)
5.  for j = 1..T_c:                                   # outer loop over K/V blocks
6.      load K_j, V_j  HBM -> SRAM
7.      for i = 1..T_r:                               # inner loop over Q blocks
8.          load Q_i, O_i, l_i, m_i  HBM -> SRAM
9.          S_ij = Q_i K_j^T                          (B_r x B_c, on chip)
10.         m~_ij = rowmax(S_ij);  P~_ij = exp(S_ij - m~_ij);  l~_ij = rowsum(P~_ij)
11.         m_i_new = max(m_i, m~_ij)
            l_i_new = e^{m_i - m_i_new} * l_i + e^{m~_ij - m_i_new} * l~_ij
12.         O_i <- diag(l_i_new)^{-1} ( diag(l_i) e^{m_i - m_i_new} O_i + e^{m~_ij - m_i_new} P~_ij V_j )   ->
HBM
13.         l_i <- l_i_new;  m_i <- m_i_new                                                    -> HBM
14.     end for
15. end for
16. return O
```

Backward pass (Section 3.1, Appendix B.4): store only $\mathbf{O}$ and the statistics (m, ℓ); recompute $\mathbf{S}, \mathbf{P}$ blockwise from Q, K, V in SRAM ("selective gradient checkpointing"); despite more FLOPs the backward is faster due to fewer HBM accesses; the $O\left(N^2\right)$ dropout mask is regenerated from saved RNG state.

Hyperparameters and setup. Block sizes set by SRAM: $B_c = \lceil M/4d \rceil$. Figure 2 middle: HBM accesses and runtime fall as block size grows; beyond about 256 runtime is limited by other factors. All benchmarks on A100.

Ablations that matter. Block size vs runtime (above). Block-sparse: LRA 2.8x vs 2.4x dense; enables 64K Path-256. Approximate-attention methods only cross over FlashAttention in speed between seq 512 and 1024; block-sparse FlashAttention fastest at all lengths; every method except Linformer OOMs before 64K, FlashAttention uses about 2x less memory than Linformer.

Code / data / license. https://github.com/HazyResearch/flash-attention (license not stated in fetched text).

Stated limitations (Section 5). Each new attention variant needs a new hand-written CUDA kernel (they ask for a Halide-like high-level language); IO-awareness beyond attention is open; single-GPU optimal within constants, multi-GPU data-transfer costs left to future work.

Use in our design. We do not implement it; we **enable** it (llama.cpp flash-attention path) so prefill at long context does not allocate an $N \times N$ score buffer, which is what frees VRAM for KV cache in 12 GB. The $O(N)$ extra-memory result (Theorem 1) is what makes the compute-buffer term in our VRAM budget (Section 2.2) roughly constant in context length; without it the buffer grows as N^2 per head during prompt processing. The online-softmax rescaling identity is also the right way to compute any streamed softmax (e.g., merging per-shard scores in the Library). PDF: https://arxiv.org/pdf/2205.14135

1.3 Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity (Fedus, Zoph, Shazeer, 2021/2022) — arXiv:2101.03961

Problem and claim. Dense models apply all parameters to every token; mixture-of-experts (MoE) selects a different subset per token, giving many parameters at roughly constant FLOPs per token, but prior MoE was complex, communication-heavy and unstable. Claim: route each token to exactly **one** expert ("switch" routing, $k = 1$), which contradicts the earlier conjecture that $k \geq 2$ is needed for router gradients, add a simple differentiable load-balancing loss and an expert capacity cap, and train stably in bfloat16 with selective float32; this gives up to 7x faster pre-training than T5-Base at equal FLOPs and scales to 1.6T parameters.

Headline results. Table 1 (quality = negative log perplexity after 100k steps; time to reach -1.50; examples/s): Switch-Base cap 1.0: -1.561, 62.8 h, 1000 ex/s; cap 1.25: -1.553, 65.0 h, 910 ex/s; cap 2.0: -1.554, 72.8 h, 860 ex/s; T5-Base -1.731, threshold not reached, 1600 ex/s; T5-Large -1.550, 131.1 h, 470 ex/s. Section 3.1: a 64-expert Switch-Base reaches T5-Base step-60k quality at step 450k, a 7.5x per-step speedup; Section 3.2: same quality in one-seventh the wall time. Switch-C (1571B, 2048 experts, 890B FLOPs/seq) is 4x faster than T5-XXL to fixed perplexity; Switch-XXL (395B, 64 experts, 6.3T FLOPs/seq) beats T5-XXL log-perplexity by 0.087 at 500k steps. Multilingual: mean 5x step speedup over mT5-Base, 91% of 101 languages at ≥4x. Sizes (Table 9): Switch-Base 7B / 128 experts / 124B FLOPs per seq; Switch-Large 26B / 128 experts / 425B. (The paper lists Switch-Base as 3.8B in Table 6 and 7.4B in Table 8; inconsistent in source.)

Method — equations.

Router (Section 2.1): logits $h(x) = W_r \cdot x$; gate value for expert i among N experts,

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_{j=1}^{N} e^{h(x)_j}}$$

Layer output with $\mathcal{T}$ the set of selected top-k indices and $E_i(x)$ expert i's computation:

$$y = \sum_{i \in \mathcal{T}} p_i(x)\, E_i(x)$$

Switch routing is Eq. (2) with $k=1$: $\mathcal{T} = \{\arg\max_i p_i(x)\}$, so $y = p_{i^*}(x)\, E_{i^*}(x)$ (the paper states $k=1$ in prose; it does not write a separate top-1 equation). The gate value keeps the router differentiable.

Expert capacity (Section 2.2):

$$\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}}\right) \times \text{capacity factor}$$

Tokens routed to an expert beyond its capacity are **dropped** for that layer and pass through the residual connection unchanged; drop rates were typically under 1%.

Load-balancing auxiliary loss added per Switch layer, T tokens in batch $\mathcal{B}$:

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\arg\max p(x) = i\} \qquad P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x)$$

f_i = fraction of tokens dispatched to expert i (non-differentiable), P_i = fraction of router probability given to expert i (differentiable). Under uniform routing $\sum_i f_i P_i = N \cdot \frac{1}{N^2} = \frac{1}{N}$, so the factor N keeps the loss scale constant in N; the loss is minimized by uniform routing. $\alpha = 10^{-2}$ (swept 10^{-1} to 10^{-5}; 10^{-2} balances load quickly without hurting the main loss).

Method — algorithm. Per Switch layer, per token: compute $h(x)$ and $p(x)$ (router in float32 even under bfloat16 — "selective precision"); pick $i^* = \arg\max_i p_i(x)$; if expert i^* has received fewer than capacity tokens this batch, send x to it and output $p_{i^*}(x)\, E_{i^*}(x)$, else output x via residual; accumulate f_i, P_i and add Eq. (4) to the loss. Mesh-TensorFlow pseudocode for router, loss and layer is in Appendix F (Figures 14–16).

Hyperparameters and setup. Pre-training: 2^{20} tokens per batch, 550k steps (≈576B tokens), C4. Fine-tuning: batch 1M, 16k steps, eval every 200. Init: truncated normal with scale reduced to 0.1x default (Table 3: at 3.5k steps, quality -2.72 ± 0.01 vs -3.60 ± 0.68 with 1.0x). Capacity factor 1.0–2.0 (1.0 fastest, 1.25 best time-to-quality). Expert dropout: 0.1 at non-expert layers, 0.4 inside experts (Table 4: GLUE 85.2, SQuAD 83.7, SuperGLUE 73.0; uniform 0.3 gives SuperGLUE 70.7). Recommendation: one expert per core; 2, 4 or 8 experts already beat dense T5-Base.

Ablations that matter. Selective precision (Table 2, 32 experts): float32 everywhere -1.718 at 1160 ex/s; bfloat16 everywhere diverges (-3.780); float32 router only -1.716 at 1390 ex/s. No-Token-Left-Behind (rerouting overflow to other experts) gave no benefit. Expert-attention layers diverged in bfloat16 and were dropped. Distillation (Tables 6–8): Switch-Base → T5-Base keeps ~30% of the gain (-1.580 vs T5-Base -1.636 and teacher -1.444) with teacher non-expert weight init and a 0.75 hard/soft loss mix; at 99% compression 28% kept.

Code / data / license. No repository link or license in fetched text (Mesh-TensorFlow pseudocode only; the public release is in T5X/Mesh-TF — not in fetched text, verify).

Stated limitations. Switch-XXL was sometimes unstable and not trained the full 1M steps; pre-training quality does not always transfer downstream (Switch-C SQuAD 87.7 vs Switch-XXL 89.6 at similar perplexity); a scaling study across data/model/expert parallelism is missing; compressed students cannot fully match the teacher.

Use in our design. We serve, not train, an MoE, so what we take is the accounting: per-token FLOPs ≈ dense FLOPs of one expert path (active parameters A), while memory and the per-token weight-read scale with the expert choice; the router and attention are small and dense (keep them on the GPU, experts in RAM). The capacity/drop mechanism does not apply at batch 1. The load-balancing intuition explains why, at inference, consecutive tokens spread over experts and why batching several tokens (speculative verification, prefill) reads many more expert bytes than a single token (quantified in Section 2.2). PDF: https://arxiv.org/pdf/2101.03961

1.4 Mixtral of Experts (Jiang et al., 2024) — arXiv:2401.04088

Problem and claim. An open-weight sparse MoE decoder that matches or beats Llama 2 70B and GPT-3.5 while using 13B active parameters per token: same architecture as Mistral 7B, but every feed-forward block is replaced by 8 SwiGLU experts with a top-2 router; 47B total parameters, 32k context, Apache 2.0.

Headline results. Table 2 (own pipeline, Mixtral 8x7B vs Llama 2 70B): MMLU 70.6 vs 69.9; HellaSwag 84.4 vs 85.4; ARC-C 59.7 vs 56.5; WinoGrande 77.2 vs 80.4; GSM8K 74.4 vs 69.6; MATH 28.4 vs 13.8; HumanEval 40.2 vs 29.3; MBPP 60.7 vs 49.8. Table 3 (includes GPT-3.5): MMLU 70.6 / 69.9 / 70.0; MBPP pass@1 60.7 / 49.8 / 52.2; GSM-8K 5-shot 58.4 / 53.6 / 57.1; MT-Bench (Instruct) 8.30 / 6.86 / 8.32. Passkey retrieval: 100% at any position within 32k; proof-pile perplexity decreases monotonically with context. Mixtral-Instruct (SFT then DPO): LMSys Arena Elo 1121 vs GPT-3.5-Turbo 1117, Claude-2.1 1117, Llama-2-70b-chat 1077.

Method — equations.

MoE layer output for n experts E_i and gating vector $G(x)$ (Eq. 1):

$$\sum_{i=0}^{n-1} G(x)_i \cdot E_i(x)$$

Gating (Eq. 2), with W_g the router weights and $\ell = x \cdot W_g \in \mathbb{R}^n$ the logits:

$$G(x) := \mathrm{Softmax}(\mathrm{TopK}\left(x \cdot W_g\right)), \qquad (\mathrm{TopK}(\ell))_i := \begin{cases} \ell_i & \text{if } \ell_i \text{ is among the top-}K \text{ coordinates of } \ell \\ -\infty & \text{otherwise} \end{cases}$$

Mixtral instance (Eq. 3), $K = 2$, experts are SwiGLU FFNs:

$$y = \sum_{i=0}^{n-1} \mathrm{Softmax}(\mathrm{Top2}\left(x \cdot W_g\right))_i \cdot \mathrm{SwiGLU}_i(x)$$

Because the softmax is taken after masking, the two selected weights sum to 1 and non-selected experts get weight 0, so only K expert FFNs are evaluated per token.

Parameter accounting: “each token has access to 47B parameters, but only uses 13B active parameters during inference.” The total is 47B, not 8 x 7B = 56B, because only the FFN blocks are replicated 8 times; attention, embeddings and norms are shared (the paper states the architecture; it does not give a per-component breakdown).

Method — algorithm. Table 1 architecture: dim 4096; n_layers 32; head_dim 128; hidden_dim 14336; n_heads 32; n_kv_heads 8 (GQA); context_len 32768; vocab_size 32000; num_experts 8; top_k_experts 2. Per layer per token: compute logits xW_g, keep top-2, softmax over the two, run the two SwiGLU experts, weighted sum. Execution: Megablocks casts the expert FFN as block-sparse matmuls so experts receiving different token counts are handled efficiently; Expert Parallelism places experts on different GPUs (with load-balancing challenges).

Hyperparameters and setup. Trained with 32k context on multilingual data; Instruct = SFT then DPO on paired feedback. Evaluation differs from the Llama 2 paper in places (hand-verified MBPP subset; TriviaQA without Wikipedia contexts).

Ablations that matter. Routing analysis (Section 5, Table 5): expert usage is similar across ArXiv, PubMed and PhilPapers (only DM Mathematics differs marginally) — **no obvious domain specialization**; routing follows syntax (e.g. code indentation tokens go to the same experts). Consecutive-token repetition of the first-choice expert: random baseline 12.5% (and ~46% for first-or-second); near random at layer 0 (~14%); 24–28% (first) and 61–67% (first-or-second) at layer 15; 20–26% and 44–53% at layer 31. This locality is what makes expert caching across adjacent tokens pay off somewhat.

Code / data / license. Apache 2.0 for both models; https://github.com/mistralai/mistral-src; Megablocks CUDA kernels contributed to vLLM; SkyPilot deployment.

Stated limitations. No dedicated section. Implied: memory scales with the 47B total, not 13B active; routing and multi-expert loading add overhead and favor batched workloads; locality can over-subscribe experts under Expert Parallelism.

Use in our design. The senior is this architecture class (top-K of n SwiGLU experts, GQA attention). Take: (a) the weight-read-per-token = shared weights + K experts per layer, used in the bandwidth bound; (b) the GQA shape (n_kv_heads 8, head_dim 128) as the worked example for KV-cache bytes; (c) the routing-locality numbers as the prior for how much speculative verification or small-batch prefill inflates expert reads. For a single user, place attention/router/embeddings on the GPU and the expert tensors in RAM computed on the CPU, never streamed across PCIe per token. PDF: https://arxiv.org/pdf/2401.04088

1.5 GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (Frantar et al., 2023) — arXiv:2210.17323

Problem and claim. Weight-only post-training quantization of 100B+ parameter models: round-to-nearest (RTN) collapses at 3 bits, and prior second-order methods (OBQ) are too slow above ~100M parameters. Claim: a one-

shot, layer-wise method using approximate second-order (Hessian) information that quantizes OPT-175B to 3–4 bits in about 4 GPU hours with negligible perplexity loss, making it runnable on a single A100, with 3-bit kernels giving 3.25x (A100) and 4.5x (A6000) end-to-end generation speedups over FP16.

Headline results. WikiText2 perplexity (FP16 / RTN / GPTQ): OPT-175B 4-bit 8.34 / 10.54 / 8.37; 3-bit 8.34 / 7.3e3 / 8.68; BLOOM-176B 4-bit 8.11 / 8.37 / 8.21; 3-bit 8.11 / 571 / 8.64. Quantization time on one A100-80GB: OPT-175B 4.2 h, BLOOM-176B 3.8 h. Grouping at 4-bit on 175B models: group 1024 costs ~0.02 extra bits and gains ~0.2 PPL; group 128 costs ~0.15 bits and gains a further ~0.1; with g=128 the gap to FP16 is 0.1–0.3 PPL. Larger models are easier to quantize (exception OPT-66B, attributed to dead units in early layers). 2-bit is stable only with small groups; ternary "reasonable" (numbers not in fetched text).

Method — equations.

Layer-wise objective for a linear layer $\mathbf{W} \in \mathbb{R}^{d_{row} \times d_{col}}$ and calibration inputs $\mathbf{X}$ ($d_{col} \times m$ for m samples):

$$\arg \min_{\widehat{\mathbf{W}}} \| \mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X} \|_2^2$$

OBQ step (quantize one weight w_q of a row and update the remaining full-precision set F), with $\mathbf{H}_F = 2\mathbf{X}_F\mathbf{X}_F^\top$:

$$w_q = \arg \min_{w_q} \frac{\left(\operatorname{quant}\left(w_q\right) - w_q\right)^2}{\left[\mathbf{H}_F^{-1}\right]_{qq}}, \qquad \boldsymbol{\delta}_F = -\frac{w_q - \operatorname{quant}\left(w_q\right)}{\left[\mathbf{H}_F^{-1}\right]_{qq}} \cdot \left(\mathbf{H}_F^{-1}\right)_{:,q}$$

Inverse-Hessian update after removing index q:

$$\mathbf{H}_{-q}^{-1} = (\mathbf{H}^{-1} - \frac{1}{\left[\mathbf{H}^{-1}\right]_{qq}} \mathbf{H}^{-1}_{:,q}\mathbf{H}^{-1}_{q,:})_{-q}$$

Lazy batch update for a block of indices Q:

$$\boldsymbol{\delta}_F = -\left(\mathbf{w}_Q - \operatorname{quant}\left(\mathbf{w}_Q\right)\right)\left(\left[\mathbf{H}_F^{-1}\right]_{QQ}\right)^{-1}\left(\mathbf{H}_F^{-1}\right)_{:,Q}$$

$$\mathbf{H}_{-Q}^{-1} = (\mathbf{H}^{-1} - \mathbf{H}^{-1}_{:,Q}\left(\left[\mathbf{H}^{-1}\right]_{QQ}\right)^{-1}\mathbf{H}^{-1}_{Q,:})_{-Q}$$

Symbols: $\operatorname{quant}(\cdot)$ rounds to the nearest grid point; $[\cdot]_{qq}$ the (q,q) entry; $(\cdot)_{:,q}$ column q; $(\cdot)_{-q}$ drop row and column q.

Three GPTQ insights over OBQ: (i) **arbitrary order** — quantizing columns in a fixed order (same for all rows) is nearly as good as OBQ's greedy order for large layers, so F and $\mathbf{H}_F^{-1}$ are shared across rows and Eq. (3) is applied once per column, not per weight; (ii) **lazy batch updates** (Eqs. 4–5) over blocks of B columns to make it compute-bound; (iii) **Cholesky reformulation** — the rows of $\mathbf{H}_{F_q}^{-1}$ needed when quantizing column q (row q from the diagonal rightward) equal the rows of the upper Cholesky factor of $\mathbf{H}^{-1}$ up to a diagonal scaling (the Cholesky factor divides row q by $\left(\left[\mathbf{H}_{F_q}^{-1}\right]_{qq}\right)^{1/2}$ instead of $\left[\mathbf{H}_{F_q}^{-1}\right]_{qq}$), so one factorization replaces all the sequential Eq. (3) updates and removes their numerical-error accumulation.

Method — algorithm. Algorithm 1, quantize $\mathbf{W}$ given $\mathbf{H}^{-1} = \left(2\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I}\right)^{-1}$ and block size B:

```
Q <- 0_{d_row x d_col}            # quantized output
E <- 0_{d_row x B}                # block quantization errors
H^-1 <- Cholesky(H^-1)^T          # upper Cholesky factor, used as the "inverse Hessian" below
for i = 0, B, 2B, ...:
    for j = i .. i+B-1:
        Q[:, j]        <- quant(W[:, j])
        E[:, j-i]      <- (W[:, j] - Q[:, j]) / H^-1[j, j]
        W[:, j:(i+B)]  <- W[:, j:(i+B)] - E[:, j-i] * H^-1[j, j:(i+B)]
    W[:, (i+B):] <- W[:, (i+B):] - E * H^-1[i:(i+B), (i+B):]
return Q
```

Hyperparameters and setup. Calibration: 128 random 2048-token segments from C4; dampening λ = 1% of the mean diagonal of $\mathbf{H}$; block size $B = 128$; per-row or grouped quantization grids (groups of 1024 or 128); the "act-order" heuristic (quantize columns by descending activation magnitude) is a repository feature and is **not in the fetched paper text**.

Ablations that matter. RTN vs GPTQ gap grows as bits fall (above); grouping buys 0.1–0.3 PPL; smaller models lose more (OPT-1.3B etc., numbers not transcribed); OPT-66B anomaly.

Code / data / license. https://github.com/IST-DASLab/gptq (code license not stated in fetched text; paper CC BY 4.0).

Stated limitations. No activation quantization; no speedup for the matmuls themselves on hardware without mixed FP16 x INT4 support (gains come from reduced memory traffic at small batch); 2-bit needs tiny groups.

Use in our design. This is the math behind “4-bit, group 128, calibration-based” — the recipe we apply after each weekly LoRA merge of the worker and for the senior once. For a single-developer system the calibration set should be the developer’s own code and prompts (128 x 2048 tokens is enough and cheap). If the serving format is GGUF/llama.cpp, the equivalent of this Hessian weighting is the importance-matrix (imatrix) path; GPTQ proper targets its own INT4 kernels. Expect ≤0.3 PPL loss at 4-bit g128 for a ≥7B model; test on held-out code. PDF: https://arxiv.org/pdf/2210.17323

1.6 AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration (Lin et al., 2024) — arXiv:2306.00978

Problem and claim. Weight-only low-bit quantization for on-device LLMs. Observation: weights are not equally important — about 1% of channels, identified by **activation** magnitude (not weight magnitude), dominate the quantization error; keeping them in FP16 nearly closes the gap but mixed precision is hard to implement. Claim: instead scale those input channels up (and the activations down) before quantization; the error on salient channels falls by about $1/s$ while the grid of the others barely changes; the scale is found by a tiny grid search on a layer-output objective, with no backprop or reconstruction, so it needs little calibration data, overfits less than GPTQ, and generalizes to instruction-tuned and multimodal models. TinyChat INT4 kernels give >3x speedup over HF FP16 on desktop and mobile GPUs. MLSys 2024 Best Paper.

Headline results. Table 1 (INT3 g128, WikiText PPL, FP16 → RTN → keep 1% FP16 by activation / by weight / random): OPT-6.7B 10.86 → 23.54 → 11.39 / 22.37 / 24.23; OPT-13B 10.13 → 46.04 → 10.43 / 48.96 / 42.00; OPT-1.3B 14.62 → 119.00 → 16.91 / 98.55 / 109.38. Table 2 (OPT-6.7B, scale s on the 1% salient channels): s = 1 / 1.25 / 1.5 / 2 / 4 → PPL 23.54 / 12.87 / 12.48 / 11.92 / 12.36; average Δ'/Δ = 1 / 1.005 / 1.013 / 1.038 / 1.213; average $\left(\Delta'/\Delta\right)\left(1/s\right)$ = 1 / 0.804 / 0.676 / 0.519 / 0.303 — best at $s=2$, worse at 4 because non-salient error grows. Table 4 (INT3 g128, WikiText2 PPL, FP16 / RTN / GPTQ / GPTQ-R / AWQ): 7B 5.47 / 6.66 / 6.43 / 6.42 / 6.24; 13B 4.88 / 5.52 / 5.48 / 5.41 / 5.32; 70B 3.32 / 3.98 / 3.88 / 3.86 / 3.74 (AWQ also best at INT4 g128). OpenFlamingo-9B COCO CIDEr 32-shot INT4 g128: FP16 81.70, RTN 77.13, GPTQ 74.98, AWQ 80.53. Calibration (Figure 8a, OPT-6.7B INT3 g128): AWQ reaches good PPL with 16 sequences vs 192 for GPTQ; distribution shift PubMed vs Enron (Figure 8b): AWQ +0.5–0.6 PPL, GPTQ +2.3–4.9. TinyChat: 3.2–3.3x average over HF FP16; up to 3.9x on RTX 4090, 3.5x on Jetson Orin; Llama-2-13B at 33 tok/s on a laptop RTX 4070 8 GB; Llama-2-70B on a 64 GB Orin.

Method — equations.

Group quantization with N bits (Eq. 1):

$$Q\left(\mathbf{w}\right) = \Delta \cdot \mathrm{Round}\left(\frac{\mathbf{w}}{\Delta}\right), \qquad \Delta = \frac{\max\left(\left|\mathbf{w}\right|\right)}{2^{N-1}}$$

Scaling an input channel by $s>1$ and inversely scaling the activation (Eq. 2):

$$Q\left(w\cdot s\right)\cdot\frac{x}{s} = \Delta'\cdot\mathrm{Round}\left(\frac{ws}{\Delta'}\right)\cdot x\cdot\frac{1}{s}$$

Error before and after (Eq. 3), $\mathrm{RoundErr}\left(\cdot\right)\in\left[0,0.5\right]$ with mean about 0.25:

$$\mathrm{Err}\left(Q\left(w\right)x\right) = \Delta\cdot\mathrm{RoundErr}\left(\frac{w}{\Delta}\right)\cdot x, \qquad \mathrm{Err}\left(Q\left(ws\right)\frac{x}{s}\right) = \Delta'\cdot\mathrm{RoundErr}\left(\frac{ws}{\Delta'}\right)\cdot x\cdot\frac{1}{s}$$

so the error ratio is $\frac{\Delta'}{\Delta}\cdot\frac{1}{s}$; because the group maximum rarely changes, $\Delta'\approx\Delta$ and the salient-channel error shrinks by about $1/s$.

Per-layer objective over the per-input-channel scale vector $\mathbf{s}$ (Eq. 4), $\mathbf{X}$ cached calibration inputs:

$$\mathbf{s}^{*} = \arg\min_{\mathbf{s}}\mathcal{L}\left(\mathbf{s}\right), \qquad \mathcal{L}\left(\mathbf{s}\right) = \left\| Q\left(\mathbf{W}\cdot\mathrm{diag}\left(\mathbf{s}\right)\right)\left(\mathrm{diag}\left(\mathbf{s}\right)^{-1}\cdot\mathbf{X}\right) - \mathbf{W}\mathbf{X}\right\|$$

Search space (Eq. 5), $\mathbf{s}_{\mathbf{X}}$ = mean activation magnitude per input channel, $\alpha\in\left[0,1\right]$:

$$\mathbf{s} = {\mathbf{s}_{\mathbf{X}}}^{\alpha}, \qquad \alpha^{*} = \arg\min_{\alpha}\mathcal{L}\left({\mathbf{s}_{\mathbf{X}}}^{\alpha}\right)$$

$\alpha=0$ is no scaling; $\alpha=1$ is the most aggressive. A grid of 20 values over [0,1] is used; weight clipping is also applied to minimize MSE (details not transcribed).

Method — algorithm.

```
for each linear layer (in forward order):
    X    <- cached inputs from calibration set
    s_X  <- mean(|X|) per input channel
    best <- (inf, None)
    for alpha in linspace(0, 1, 20):
        s   <- s_X ** alpha
        W_q <- Q(W * diag(s))              # group quantization, e.g. INT4 g128
        err <- || W_q (diag(s)^-1 X) - W X ||
        if err < best.err: best <- (err, s)
    fold diag(best.s)^-1 into the preceding op (LayerNorm / previous linear) so runtime cost is zero
    store W_q = Q(W * diag(best.s))
```

Hyperparameters and setup. INT3/INT4 with group size 128; a small calibration set from the Pile (16 sequences suffice per Figure 8a; the exact “16 x 512” figure is not in the fetched text); α grid of 20; TinyChat benchmarks at batch 1, 4-token prompt, 200 generated tokens, median latency; W4A16 throughput (Table 10, VILA-7B): A100 81.6 → 155.3, RTX 4090 58.5 → 168.1, Orin 11.5 → 35.6.

Ablations that matter. Selection criterion: activation-based 1% FP16 beats weight-based and random by 10–40 PPL at INT3 (Table 1). Scale magnitude: $s = 2$ optimal in isolation (Table 2). Calibration size and distribution robustness (Figure 8). INT2: still poor even combined with GPTQ (OPT-1.3B 35.71 vs 14.62 FP16).

Code / data / license. https://github.com/mit-han-lab/llm-awq (license not stated in fetched text).

Stated limitations. Mixed-precision (FP16 salient channels) is accurate but hard to implement efficiently — motivation for scaling; INT2 remains hard; evaluation leans on WikiText-2 PPL plus a few tasks and GPT-4 judging.

Use in our design. AWQ’s two facts drive our quantization policy: salience is an **activation** property, so calibrate on the developer’s real code/prompt traffic (few sequences needed, robust to shift, and our weekly LoRA-merged worker can be re-quantized in minutes); and per-channel scaling folds into the previous layer, so INT4 g128 costs nothing at runtime. In GGUF land the same activation statistics feed the imatrix; if we use a non-GGUF 4-bit path for the GPU worker, AWQ INT4 g128 is the default. Expect AWQ ≈ 0.1–0.2 PPL better than GPTQ at 3-bit and roughly tied at 4-bit. PDF: https://arxiv.org/pdf/2306.00978

1.7 Lost in the Middle: How Language Models Use Long Contexts (Liu et al., 2023) — arXiv:2307.03172

Problem and claim. Do language models actually use information anywhere in a long input? Two controlled tasks: multi-document QA (NaturalQuestions-Open, Contriever-retrieved Wikipedia distractors plus one gold document whose position is varied) and synthetic key-value retrieval (JSON of random UUID pairs, 75/140/300 pairs). Claim: performance is a **U-shaped function of position** — highest when the relevant information is at the beginning (primacy) or end (recency) and lowest in the middle, where it can fall **below the closed-book (no documents) baseline**; this holds for models explicitly built for long context, and the U-shape strengthens with scale.

Headline results. Table 1 baselines (closed-book / oracle = only the gold document): GPT-3.5-Turbo 56.1 / 88.3; GPT-3.5-Turbo-16K 56.0 / 88.6; Claude-1.3 48.3 / 76.1; Claude-1.3-100K 48.2 / 76.4; MPT-30B-Instruct 31.5 / 81.9; LongChat-13B-16K 35.0 / 83.4.

20 documents (Table 6; gold at 0-based index):

Model	idx 0	idx 4	idx 9	idx 14	idx 19
GPT-3.5-Turbo	75.8	57.2	53.8	55.4	63.2
GPT-3.5-Turbo-16K	75.7	57.3	54.1	55.4	63.1
Claude-1.3	59.9	55.9	56.8	57.2	60.1
Claude-1.3-100K	59.8	55.9	57.0	57.4	60.0
MPT-30B-Instruct	53.7	51.8	52.2	52.7	56.3
LongChat-13B-16K	68.6	57.4	55.3	52.5	55.0

GPT-3.5-Turbo at idx 9 (53.8) is below its closed-book 56.1; best-to-worst gap 22.0 points (computed from the table). 10 documents (Table 5, idx 0/4/9): GPT-3.5-Turbo 76.8/61.2/62.4; Claude-1.3 62.9/58.3/59.7; MPT-30B-Instruct 60.2/56.2/59.7; LongChat-13B-16K 72.1/58.9/58.5. 30 documents (Table 7, idx 0/4/9/14/19/24/29): GPT-3.5-Turbo-16K 73.4/55.1/50.5/50.9/51.8/54.9/63.7; Claude-1.3 59.1/55.1/54.8/55.7/56.4/56.2/59.9; MPT-30B-Instruct 51.6/51.3/51.2/49.0/49.6/51.3/54.1; LongChat-13B-16K 66.9/54.8/52.5/52.9/52.2/51.3/55.1. Prompt lengths (Appendix F, GPT-3.5-Turbo tokenizer): 1475.6 / 2946.2 / 4419.2 tokens at 10 / 20 / 30 documents. Key-

value retrieval: Claude-1.3 near-perfect at 75/140/300 pairs; others degrade at 140 and 300; worst case without query-aware contextualization 45.6%; with the query placed before **and** after the data all models become near-perfect (exact per-model figures only in Figure 7, not in fetched text).

Method — equations. None; the protocol is the method. Accuracy = whether any gold answer string appears in the greedy-decoded output. Position index $i \in \{0, ..., k-1\}$ of the gold document among k retrieved; report accuracy as a function of i and the best-minus-worst gap.

Method — algorithm. Multi-document QA construction:

```
for each NQ-Open question q with gold Wikipedia paragraph g:
    D <- top-k Contriever (MS-MARCO fine-tuned) passages for q, excluding any that contain a gold answer
    for each test position i in {0, ..., k-1}:
        ctx <- D with g inserted at index i (k documents total)
        prompt <- instruction + ctx + question
        acc[i] += gold_answer_substring(greedy(model, prompt))
report acc[i] per model; closed-book (no ctx) and oracle (ctx = [g]) baselines
```

Key-value task: JSON of n UUID→UUID pairs, ask for the value of one key at varied position. Query-aware contextualization = put the question both before and after the documents/JSON.

Hyperparameters and setup. $k \in \{10, 20, 30\}$; greedy decoding; models above plus Llama-2 7B/13B/70B (base and chat), Flan-T5-XXL, Flan-UL2, GPT-4 on a 500-example subset (cost).

Ablations that matter. Query-aware contextualization: near-perfect key-value retrieval; on multi-document QA it only slightly helps at idx 0 and slightly hurts elsewhere. Instruction tuning: MPT-30B base and Instruct both U-shaped; tuning raises absolute accuracy and shrinks the gap from ~10 to ~4 points. Scale (Llama-2, Appendix E): 7B is recency-biased only; 13B and 70B show the U, 13B base with a 20-point gap. Encoder-decoders: Flan-UL2 nearly flat (~1.9-point gap) within its 2048-token training length, U-shaped beyond it. Retriever-reader (Section 5): going from 20 to 50 retrieved documents adds only ~1.5% (GPT-3.5-Turbo) and ~1% (Claude-1.3) despite rising recall — reader saturates before retriever recall does.

Code / data / license. Project page nelsonliu.me/papers/lost-in-the-middle (the GitHub URL is not on the fetched page — verify).

Stated limitations. Greedy decoding only; NQ-Open ambiguity (an unambiguous subset gave the same picture); slight temporal mismatch between the Wikipedia dump and NQ; GPT-4 only on 500 examples; 20 Llama-2 examples dropped for exceeding 4096 tokens.

Use in our design. This is the justification for the Library's **hard context budget** and **position-aware packing**: a small worker (9B, between the 7B "recency-only" and 13B "U-shaped" regimes) should get few, highly ranked chunks, with the top-ranked chunk first and the second-ranked last, the instruction/question repeated after the evidence, and no more than ~10–20 chunks regardless of retriever recall. Evaluate our own models with the same protocol (gold chunk at first / middle / last) before trusting any long-context claim. PDF: https://arxiv.org/pdf/2307.03172

1.8 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020) — arXiv:2005.11401

Problem and claim. Parametric seq2seq models store knowledge in weights that are hard to inspect, update or extend. Claim: combine a pre-trained seq2seq generator (BART-large) with a non-parametric dense index (DPR over Wikipedia), treat the retrieved passage as a **latent variable** marginalized at training and decoding time, fine-tune retriever (query side) and generator jointly end-to-end with only the final output as supervision, and get state of the art on three open-domain QA sets plus more factual, specific generation. The index can be hot-swapped to update knowledge without retraining.

Headline results. Table 1 (exact match, test): NQ / TriviaQA / WebQuestions / CuratedTrec — T5-11B closed-book 34.5 / 50.1 / 37.4 / –; REALM 40.4 / – / 40.7 / 46.8; DPR 41.5 / 57.9 / 41.1 / 50.6; RAG-Token 44.1 / 55.2 / 45.5 / 50.0; RAG-Sequence 44.5 / 56.8 / 45.2 / 52.2 (TQA-Wiki split: 66.1 / 68.0). Table 2: Jeopardy question generation B-1/QB-1 BART 15.1/19.7, RAG-Token 17.3/22.2; MS-MARCO B-1/R-L BART 41.6/38.2, RAG-Sequence 44.2/40.8; FEVER 3-way/2-way BART 64.0/81.1, RAG-Token 72.5/89.5 (SOTA with retrieval supervision 76.8/92.2). Index hot-swap: 2016 vs 2018 Wikipedia, world-leader questions ~70%/68% correct with the matching index, 12%/4% with the mismatched one.

Method — equations.

Retriever (DPR bi-encoder), $\mathbf{d}(z) = \mathrm{BERT}_d(z)$, $\mathbf{q}(x) = \mathrm{BERT}_q(x)$:

$$p_\eta\left(z \mid x\right) \propto \exp\left(\mathbf{d}\left(z\right)^\top \mathbf{q}\left(x\right)\right)$$

Top-k via maximum inner product search (MIPS, sub-linear with FAISS HNSW).

RAG-Sequence — one document for the whole output, marginalized over the top-k:

$$p_{\text{RAG-Sequence}}\left(y \mid x\right) \approx \sum_{z \in \text{top-}k\left(p\left(\cdot \mid x\right)\right)} p_\eta\left(z \mid x\right) p_\theta\left(y \mid x, z\right) = \sum_{z \in \text{top-}k\left(p\left(\cdot \mid x\right)\right)} p_\eta\left(z \mid x\right) \prod_{i=1}^{N} p_\theta\left(y_i \mid x, z, y_{1:i-1}\right)$$

RAG-Token — a different document may be used per token (the paper writes the retriever term with a per-token index, $p_\eta\left(z_i \mid x\right)$):

$$p_{\text{RAG-Token}}\left(y \mid x\right) \approx \prod_{i=1}^{N} \sum_{z \in \text{top-}k\left(p\left(\cdot \mid x\right)\right)} p_\eta\left(z \mid x\right) p_\theta\left(y_i \mid x, z, y_{1:i-1}\right)$$

Training: minimize the negative marginal log-likelihood $\sum_j -\log p\left(y_j \mid x_j\right)$ over pairs $\left(x_j, y_j\right)$ with Adam; **document encoder and index frozen**, only BERT_q and the generator θ updated (refreshing the index during training was costly and not needed).

Decoding. RAG-Token: standard beam search with the per-step transition

$$p'_\theta\left(y_i \mid x, y_{1:i-1}\right) = \sum_{z \in \text{top-}k\left(p\left(\cdot \mid x\right)\right)} p_\eta\left(z \mid x\right) p_\theta\left(y_i \mid x, z, y_{1:i-1}\right)$$

RAG-Sequence: the likelihood does not factor per token, so run beam search per document z, collect the union of hypotheses Y, then **Thorough Decoding** — for every $y \in Y$ and every z whose beam did not produce y, run an extra forward pass to get $p_\theta\left(y \mid x, z\right)$, then score $p\left(y \mid x\right) \approx \sum_z p_\eta\left(z \mid x\right) p_\theta\left(y \mid x, z\right)$; **Fast Decoding** — set $p_\theta\left(y \mid x, z\right) \approx 0$ for y not in z's beam, skipping the extra passes.

Symbols: x input, y output of N tokens, z retrieved passage (latent), η retriever params, θ generator params, k number of retrieved passages.

Method — algorithm. Index: Wikipedia Dec-2018, 100-word disjoint chunks, 21M passages, DPR document encoder, FAISS HNSW (about 15.3B values, storable at 8-bit; ~100 GB CPU RAM, 36 GB compressed). Per example: encode x, MIPS top-k, prepend each z to x, run BART per z, combine per the chosen model.

Hyperparameters and setup. BART-large (406M) generator; DPR BERT-base encoders; $k = 5$ or 10 in training; test k chosen on dev — 15 for RAG-Token QA, 50 for RAG-Sequence QA, 10 for MS-MARCO and Jeopardy; Fairseq, mixed precision, 8 x V100 32 GB; learning rates not in fetched text.

Ablations that matter. Freezing the retriever hurts everywhere (NQ dev EM 43.5 → 37.8 RAG-Token, 44.0 → 41.2 RAG-Sequence). BM25 instead of DPR is worse on QA but slightly better on FEVER (entity-heavy claims). More documents (Figure 3): RAG-Sequence NQ keeps improving to 50; RAG-Token peaks at 10; on MS-MARCO more documents raise Rouge-L but lower Bleu-1.

Code / data / license. HuggingFace Transformers RAG example (github.com/huggingface/transformers, examples/rag). Code license not stated in fetched text.

Stated limitations. Retrieval collapse on some tasks (story generation: retriever returns the same documents regardless of input and performance falls back to BART); hallucination reduced, not eliminated; Wikipedia is not fully factual or unbiased; misuse risks as for GPT-2.

Use in our design. Take the latent-document framing: retrieval provides a **distribution** over chunks, not one answer; the Library returns scored candidates and the generator is conditioned on several. We do **not** fine-tune the retriever end-to-end (no supervision, single user); we use a frozen dense encoder plus BM25 as the hybrid first stage and move the "which document matters" decision to the reranker. The "number of documents" ablation (RAG-Token peaks at 10) and the index hot-swap result (knowledge lives in the index — re-index on every repo change) carry over directly. PDF: https://arxiv.org/pdf/2005.11401

1.9 ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT (Khattab and Zaharia, 2020) — arXiv:2004.12832

Problem and claim. BERT cross-encoders rerank well but cost ~10–30 s per query over 1000 candidates; single-vector bi-encoders are fast but lose fine-grained matching. Claim: encode queries and documents **independently** into bags of token-level embeddings (precompute documents offline), then score with a cheap **late interaction** — for each query token take the maximum similarity over document tokens and sum (MaxSim). This keeps most of the cross-encoder quality at ~170x lower latency and ~14,000x fewer FLOPs, and because

MaxSim decomposes into per-token nearest-neighbour searches it supports end-to-end retrieval from the full collection with a vector index.

Headline results. Table 1, MS MARCO passage re-ranking of BM25 top-1000 (MRR@10 dev / eval, latency ms, FLOPs per query): BM25 16.7 / 16.5; KNRM 19.8 / 19.8, 3 ms, 592M; Duet 24.3 / 24.5, 22 ms, 159B; fastText+ConvKNRM 29.0 / 27.7, 28 ms, 78B; BERT-base 34.7 / –, 10,700 ms, 97T; BERT-large 36.5 / 35.9, 32,900 ms, 340T; **ColBERT (BERT-base) 34.9 / 34.9, 61 ms, 7B**. Table 2, end-to-end retrieval on MS MARCO (MRR@10, R@50, R@200, R@1000): BM25 16.7, –, –, 81.4; doc2query 21.5, 64.4, 77.9, 89.1; DeepCT 24.3, 69, 82, 91; docTTTTTquery 27.7, 75.6, 86.9, 94.7; ColBERT-L2 re-rank 34.8, 75.3, 80.5, 81.4; **ColBERT-L2 end-to-end 36.0, 82.9, 92.3, 96.8**. TREC CAR MAP: BM25 15.3, DeepCT 24.6, BM25+BERT-base 31.0, BM25+BERT-large 33.5, BM25+ColBERT 31.3 (MRR@10 44.3).

Method — equations.

Query encoder (Eq. 1; "#" are [mask] tokens appended up to length N_q — query augmentation; the text describes the projection as a linear layer without activation although the equation writes CNN):

$$E_q := \text{Normalize}(\text{CNN}(\text{BERT}\left(\text{"[Q]}\ q_0 q_1 ... q_l\ \#...\#\text{"}\right)))$$

Document encoder (Eq. 2; Filter drops embeddings of punctuation tokens from a fixed list; no [mask] padding):

$$E_d := \text{Filter}(\text{Normalize}(\text{CNN}(\text{BERT}\left(\text{"[D]}\ d_0 d_1 ... d_n\text{"}\right))))$$

Late interaction, MaxSim (Eq. 3):

$$S_{q,d} := \sum_{i \in \left[\left|E_q\right|\right]} \max_{j \in \left[\left|E_d\right|\right]} E_{q_i} \cdot E_{d_j}^{\top}$$

Symbols: q query of l WordPiece tokens, d document of $n+1$ tokens; [Q], [D] special tokens placed after [CLS]; N_q fixed padded query length ($\left|E_q\right| = N_q$); m output dimension of the projection (much smaller than BERT's hidden size); Normalize = L2, so dot products are cosines (an L2-distance variant "ColBERT-L2" is used for end-to-end retrieval); E_{q_i}, E_{d_j} the i-th query and j-th document embedding. Training: pairwise softmax cross-entropy over the scores of d^+ and d^- for each triple $\langle q, d^+, d^- \rangle$ (no explicit formula in the paper).

Method — algorithm.

```
offline:
    for each document d: E_d <- Filter(Normalize(Linear(BERT([CLS][D] d))))   # n_d x m, store (16- or 32-bit)
    build faiss IVFPQ over all document token embeddings; keep map embedding -> doc id
re-rank (candidates C from BM25):
    E_q <- Normalize(Linear(BERT([CLS][Q] q [mask]*)))                        # N_q x m
    for d in C: S_qd <- sum_i max_j E_q[i] . E_d[j]                            # batched on GPU
    return top-k by S_qd
end-to-end:
    for each of the N_q query embeddings: retrieve top-k' nearest document embeddings from the index
    K <- unique doc ids among the N_q * k' hits   (K <= N_q * k')
    re-rank the K documents exhaustively with Eq. (3); return top-k
```

Hyperparameters and setup. lr 3×10^{-6}, Adam, batch 32, 200k iterations on MS MARCO (125k on TREC CAR); $N_q = 32$; $m = 128$; documents padded to the longest in the batch; IVFPQ with $P = 2000$ partitions, nprobe $p = 10$, $s = 16$ one-byte sub-vectors per embedding; $k' = k = 1000$ per query embedding in experiments ($k' = k/2$ given as an example in the method section). Space (Table 4, MS MARCO 8.8M passages): $m = 128$ at 4 bytes/dim 286 GiB (MRR 34.9); 128 at 2 bytes 154 GiB end-to-end (36.0) / 143 GiB re-rank (34.8); $m = 48$ at 4 bytes 54 GiB (34.4); $m = 24$ at 2 bytes 27 GiB (33.9).

Ablations that matter. Dimension and precision (Table 4): going from 128 to 24 dims and 2 bytes cuts storage 10x for 1.0 MRR. End-to-end retrieval beats re-ranking BM25 (36.0 vs 34.8) because BM25 recall caps the re-ranker (R@1000 81.4). Latency scales with candidate count and is dominated by BERT query encoding plus batched MaxSim on GPU.

Code / data / license. No repository URL in the fetched text (the canonical repo is github.com/stanford-futuredata/ColBERT — verify); license not stated.

Stated limitations. No dedicated section; deferred: shorter query padding, quantization of embeddings, GPU-resident embeddings, CPU re-ranking. Storage is the practical cost (hundreds of GiB for 8.8M passages at full precision).

Use in our design. MaxSim is the Library's reranker of choice over a cross-encoder for a single-user corpus: document token embeddings are precomputed per chunk at index time (cheap at our scale — a few hundred

thousand tokens of code and docs, i.e. tens of MB at $m = 128$, 2 bytes), so reranking the hybrid top-100 is a few matrix ops, and the per-token maxima tell us **which tokens matched**, which we use to trim chunks to the matching region inside the context budget. Use BM25 + dense top-k as the candidate set rather than ColBERT end-to-end (no IVFPQ needed at our scale). A cross-encoder remains the fallback when a ColBERT checkpoint for code is unavailable. PDF: https://arxiv.org/pdf/2004.12832

2. Design extraction for the module

2.1 Algorithms to implement (consolidated pseudocode)

A. Speculative decoding step (Leviathan et al., Alg. 1) — used as-is for the worker; configuration only if llama.cpp provides it.

```
function SPEC_STEP(target M_p, draft M_q, prefix, gamma):
    drafts = [];  qs = []
    for i in 1..gamma:
        q_i = M_q.next_dist(prefix + drafts)          # adjusted for temperature/top-k identically to target
        x_i ~ q_i;  drafts.append(x_i);  qs.append(q_i)
    ps = M_p.dists_parallel(prefix, drafts)            # p_1..p_{gamma+1} in one forward pass
    n = gamma
    for i in 1..gamma:
        r ~ U(0,1)
        if r > ps[i][x_i] / qs[i][x_i]:  n = i - 1;  break
    if n < gamma: p_adj = normalize(max(0, ps[n+1] - qs[n+1]))
    else:         p_adj = ps[gamma+1]
    t ~ p_adj
    return prefix + drafts[0:n] + [t]                  # roll back draft/target KV caches to prefix+n first
```

B. Choosing gamma from measured alpha and c.

```
function CHOOSE_GAMMA(alpha, c, gamma_max=10):
    best = argmax_{g in 1..gamma_max} (1 - alpha^(g+1)) / ((1 - alpha) * (g*c + 1))
    return best
# alpha: fraction of draft tokens accepted over >= 10K generated tokens on real traffic (Leviathan Sec. 3.1)
# c: measured draft step time / target step time
# only enable if alpha > c (Corollary 3.9)
```

C. GPTQ layer quantization (Frantar et al., Alg. 1) — for re-quantizing the worker after a LoRA merge (or drive the serving framework's equivalent).

```
function GPTQ_LAYER(W [d_row x d_col], X [d_col x m], bits, group=128, B=128, damp=0.01):
    H = 2 * X @ X.T
    H += damp * mean(diag(H)) * I
    Hinv = cholesky_upper(inverse(H))                  # Cholesky(H^-1)^T
    Q = zeros_like(W);  E = zeros(d_row, B)
    for i in range(0, d_col, B):
        for j in range(i, i+B):
            if j % group == 0: (scale, zero) = fit_grid(W[:, j:j+group], bits)   # grid per group of 128
columns
            Q[:, j]       = quant(W[:, j], scale, zero)
            E[:, j-i]     = (W[:, j] - Q[:, j]) / Hinv[j, j]
            W[:, j:i+B]  -= outer(E[:, j-i], Hinv[j, j:i+B])
        W[:, i+B:] -= E @ Hinv[i:i+B, i+B:]
    return Q, scales, zeros
# calibration X: 128 sequences x 2048 tokens from the developer's own code and prompts (paper used C4)
# layers processed sequentially so X for layer k is the quantized output of layers < k
```

D. AWQ scale search (Lin et al., Eq. 4–5) — alternative/complement; also the recipe for building an activation-importance file.

```
function AWQ_LAYER(W, X, bits, group=128, n_grid=20):
    sX = mean(abs(X), axis=tokens)                    # per input channel
    best = (inf, None)
    for alpha in linspace(0, 1, n_grid):
        s = sX ** alpha
        err = norm(quant_group(W * s[None, :], bits, group) @ (X / s[:, None]) - W @ X)
        if err < best[0]: best = (err, s)
    fold 1/s into the preceding LayerNorm or linear layer
    return quant_group(W * best[1][None, :], bits, group)
```

E. Memory and bandwidth budgeting (derivation, Section 2.2) — run at startup and whenever a model or context setting changes.

```
function PLAN_SERVING(model, quant, device_mem, kv_bits, draft):
    W   = model.params_total * bpw(quant) / 8
    KVt = 2 * model.layers * model.n_kv_heads * model.head_dim * kv_bits / 8
    buf = compute_buffer_estimate(model, flash_attention=true)      # roughly constant in n_ctx with
FlashAttention
    D   = draft ? draft.params * bpw(draft.quant) / 8 : 0
    n_ctx_max = floor((device_mem - W - buf - D) / KVt)
    bytes_per_token(n) = active_bytes + n * KVt                     # active_bytes = W for dense, A*bpw/8
(+shared) for MoE
    tok_s_bound(n) = eta * bandwidth / bytes_per_token(n)
    return {W, KVt, n_ctx_max, tok_s_bound}
```

F. Library indexing (RAG/DPR + BM25 + ColBERT).

```
function INDEX_REPO(root):
    chunks = []
    for file in walk(root, include=[code, md, docs], exclude=[.git, node_modules, build]):
        for ch in split(file):            # code: by top-level symbol then by ~100-200 tokens with 20% overlap;
docs: by heading then ~100 words (RAG used 100-word chunks)
            chunks.append(Chunk(id, path, span, text, lang, symbol, hash))
    bm25   = BM25Index(tokenize_code_aware(c.text) for c in chunks)         # identifiers split on
camelCase/snake_case, keep originals too
    dense  = DenseIndex(encode_doc(c.text) for c in chunks)                 # single vector per chunk (DPR-
style), cosine
    colb   = ColbertIndex(filter_punct(normalize(linear(encoder(c.text)))) for c in chunks)  # token matrices,
fp16, m=128 or smaller
    persist(chunks, bm25, dense, colb); watch(root) -> incremental re-index of changed files by hash
```

G. Retrieve, rerank, pack under a hard budget (hybrid first stage; MaxSim rerank; Lost-in-the-Middle-aware packing).

```
function RETRIEVE(query, k1=50, k_final=8, budget_tokens):
    C = union(bm25.top(query, k1), dense.top(encode_query(query), k1))     # hybrid candidates
    Eq = normalize(linear(encoder("[Q] " + query + " [mask]"*pad_to_Nq)))  # N_q = 32
    for c in C: c.score, c.match = maxsim(Eq, colb[c.id])                   # sum_i max_j Eq[i].Ed[j]; keep
argmax_j per i
    R = top(C, by=score, k=k_final)
    return PACK(R, budget_tokens)

function PACK(R, budget):
    # Lost in the Middle: best positions are first and last; middle is worst; fewer docs beyond ~10-20 add
little
    R = trim_each_to_matching_region(R, window_around(c.match))             # use MaxSim argmax positions
    # rank 1 first, rank 2 last, remaining ranks alternate inward so the weakest chunks sit in the middle:
    # slots = [R[0], R[2], R[4], ..., R[5], R[3], R[1]]
    front = [R[i] for i in range(0, len(R), 2)]
    back  = [R[i] for i in range(1, len(R), 2)][::-1]
    slots = front + back
    ctx = []; used = 0
    for c in slots:
        if used + len(c) > budget: continue          # hard budget; drop, never truncate mid-symbol
        ctx.append(c); used += len(c)
    return header(question) + ctx + footer(question)  # query-aware contextualization: question before AND
after evidence
```

2.2 Equations and quantities the implementation must compute

From the papers (transcribed; see Section 1 for symbols).

1. Acceptance test per draft token: accept iff $r \leq \min\left(1, p(x)/q(x)\right)$; residual $p'(x) = \text{norm}(\max(0, p(x) - q(x)))$. (1.1)
2. Expected tokens per target call $\frac{1-\alpha^{\gamma+1}}{1-\alpha}$; wall-time factor $\frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}$; ops factor $\frac{(1-\alpha)(\gamma\hat{c}+\gamma+1)}{1-\alpha^{\gamma+1}}$; $\alpha = E(\min(p, q))$. (1.1)
3. Online softmax merge: $m = \max(m_1, m_2)$, $\ell = e^{m_1 - m}\ell_1 + e^{m_2 - m}\ell_2$, $O \leftarrow \text{diag}(\ell^{new})^{-1}\left(\text{diag}(\ell)\, e^{m - m^{new}} O + e^{\tilde{m} - m^{new}} \tilde{P} V\right)$; HBM accesses $\Theta\left(N^2 d^2 M^{-1}\right)$ vs $\Theta\left(Nd + N^2\right)$; extra memory $O(N)$. (1.2)
4. MoE gate: Switch $p_i(x) = \text{softmax}(W_r x)_i$, $y = \sum_{i \in \mathcal{T}} p_i E_i(x)$ with $|\mathcal{T}| = 1$; Mixtral $y = \sum_i \text{Softmax}\left(\text{Top2}\left(xW_g\right)\right)_i \text{SwiGLU}_i(x)$; load loss $\alpha N \sum_i f_i P_i$, $\alpha = 10^{-2}$; capacity $= \frac{\text{tokens}}{\text{experts}} \times \text{CF}$. (1.3, 1.4)
5. GPTQ: objective $\|WX - \widehat{W}X\|_2^2$; $H = 2XX^\top + \lambda I$; per-column error $E = (w - q)/\left[H^{-1}\right]_{jj}$ with H^{-1} replaced by its upper Cholesky factor; propagation $W_{:,j:} -= E H^{-1}_{j,j:}$. (1.5)
6. AWQ: $\Delta = \max|w|/2^{N-1}$; error ratio $\frac{\Delta' 1}{\Delta s}$; $\mathcal{L}(s) = \|Q(W\text{diag}(s))\left(\text{diag}(s)^{-1}X\right) - WX\|$ over $s = s_X^\alpha$, $\alpha \in [0, 1]$, 20-point grid. (1.6)
7. RAG: $p(y|x) \approx \sum_{z \in \text{top-}k} p_\eta(z|x)\, p_\theta(y|x, z)$ (sequence) or $\prod_i \sum_z p_\eta(z|x)\, p_\theta\left(y_i|x, z, y_{<i}\right)$ (token); $p_\eta(z|x) \propto \exp\left(d(z)^\top q(x)\right)$. We use the retriever distribution only as a candidate scorer; the generator sees the packed chunks. (1.8)
8. ColBERT MaxSim: $S_{q,d} = \sum_{i=1}^{N_q} \max_j E_{q_i} \cdot E_{d_j}^\top$ with L2-normalized m-dim embeddings. (1.9)
9. Position-sensitivity: report accuracy at gold index 0, mid, $k-1$ and the best–worst gap; compare the middle against the closed-book baseline. (1.7)

Memory and bandwidth arithmetic — derivations, not paper results. Assumptions: (A1) single-stream decode is memory-bound: every byte of weight that participates in a token is read once from the memory that holds it per generated token; (A2) compute overlaps bandwidth perfectly; (A3) quantized tensors are block formats with b bits per weight plus per-block metadata; (A4) bandwidths are nominal peaks times an achieved fraction $\eta \in [0.6, 0.8]$; (A5) numbers for a specific “9B worker” shape are illustrative — substitute the real layer count, KV heads and head dimension of the chosen model.

a. Bytes per parameter. For block size g weights, a per-block scale of s bits and optional per-block offset/minimum of o bits:

$$\text{bpw}_{\text{eff}} = b + \frac{s + o}{g}, \qquad \text{bytes per parameter} = \frac{\text{bpw}_{\text{eff}}}{8}$$

Nominal: Q4 = 0.500, Q6 = 0.750, Q8 = 1.000, F16 = 2.000 bytes. Checked against llama.cpp’s documented values (tools/quantize README, “Llama 2 7B” list; fetched from a mirror): Q4_0 ($b = 4, g = 32, s = 16$) → 4.5 bpw ✓; Q4_1 ($o = 16$) → 5.0 ✓; Q8_0 ($b = 8, g = 32, s = 16$) → 8.5 ✓; Q6_K (super-block 256 = 16 sub-blocks with 8-bit sub-scales + one 16-bit super-scale: $6 + (16 \cdot 8 + 16)/256 = 6.5625$) → documented 6.56 ✓; Q4_K_S 4.58, Q4_K_M 4.84, Q5_K_M 5.68 (mixed per-tensor precisions; take the documented numbers). Weight memory $W = P_{\text{total}} \cdot \text{bpw}_{\text{eff}}/8$:

Model	Q4_K_M (4.84)	Q5_K_M (5.68)	Q6_K (6.56)	Q8_0 (8.5)	F16
9B dense	5.45 GB (5.07 GiB)	6.39 GB	7.38 GB	9.56 GB	18.0 GB
35B-total MoE	21.2 GB (19.7 GiB)	24.9 GB	28.7 GB	37.2 GB	70 GB

Fit: senior at Q4_K_M in 32 GB RAM leaves ~10 GB for OS, KV and buffers; Q6_K leaves ~3 GB (tight; OS typically needs 2–4 GB); Q8_0 does not fit. Worker at Q4_K_M in 12 GB VRAM leaves ~6 GB; Q6_K ~4 GB; Q8_0 ~2 GB — to be shared by KV cache, compute buffers and the draft model (a 0.5–1B draft at Q8_0 is 0.5–1.1 GB).

b. KV-cache bytes per token, L layers, n_{kv} KV heads, d_{head} head dim, b_{kv} bits per cached element (16 for f16; 8.5 for q8_0; 4.5 for q4_0 block formats):

$$\text{KV}_{\text{tok}} = 2 \cdot L \cdot n_{kv} \cdot d_{head} \cdot \frac{b_{kv}}{8}$$

Worked example on Mixtral’s verified shape (Table 1 of 1.4: $L = 32$, $n_{kv} = 8$, $d_{head} = 128$): f16 → $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131{,}072$ B = 128 KiB per token → 4 GiB at 32,768 tokens; q8_0 → 68 KiB/token → 2.1 GiB at 32k; without GQA ($n_{kv} = 32$) it would be 512 KiB/token → 16 GiB at 32k. Illustrative worker shape (A5: $L = 42$, $n_{kv} = 8$, $d_{head} = 128$): 168 KiB/token f16 → 2.6 GiB at 16k, 5.25 GiB at 32k. Maximum context that fits:

$$n_{ctx}^{\max} = \left\lfloor \frac{\mathrm{Mem}_{\mathrm{device}} - W - W_{\mathrm{draft}} - \mathrm{Buf}}{\mathrm{KV}_{\mathrm{tok}}} \right\rfloor$$

e.g. 11.2 GiB usable − 5.07 (Q4_K_M) − 0.7 (draft) − 1.0 (buffers, roughly constant with FlashAttention) ≈ 4.4 GiB → ~27k tokens at f16 or ~52k at q8_0 for the illustrative shape. The attention step reads the whole cache, so per-token bytes grow with context: $\mathrm{bytes}_{\mathrm{tok}}(n) = W_{\mathrm{active}} + n \cdot \mathrm{KV}_{\mathrm{tok}}$; at 32k with 128 KiB/token the KV read (4 GiB) is comparable to the active weights — this is why KV quantization and GQA matter as much as weight quantization at long context.

c. Tokens/s bound by memory bandwidth, for an MoE with A active parameters whose experts live in RAM:

$$\mathrm{tok/s} \le \frac{\eta \cdot BW}{A \cdot \mathrm{bpw}_{\mathrm{eff}}/8 + W_{\mathrm{shared}} + n \cdot \mathrm{KV}_{\mathrm{tok}}}$$

where W_{shared} (attention, router, embeddings) and the KV term are charged to whichever device holds them; if the GPU and CPU phases do not overlap, add the two times. Worked: $A = 3 \times 10^9$ at Q4_K_M → 1.815 GB read from RAM per token. Dual-channel DDR5-5600: $2 \cdot 8\ \mathrm{B} \cdot 5600 \times 10^6 = 89.6$ GB/s peak, $\eta = 0.7$ → 62.7 GB/s → **≤ 34.5 tok/s** (experts only). Dual-channel DDR4-3200: 51.2 GB/s peak → 35.8 → **≤ 19.7 tok/s**. Adding a 4 GiB KV read on a 360 GB/s GPU (RTX 3060-class) at 32k context, serial: $1/(0.029 + 0.012) \approx 24$ tok/s. Dense worker fully in VRAM: 5.45 GB/token; at 360 GB/s × 0.8 → **≤ 53 tok/s**; at 504 GB/s × 0.8 (RTX 4070-class) → ≤ 74 tok/s. PCIe caveat: if expert weights were copied to the GPU per token over PCIe 4.0 x16 (~32 GB/s peak) the bound falls to ≤ 12 tok/s — so experts must be **computed where they live** (CPU), not streamed (llama.cpp's tensor-placement override keeping expert tensors on CPU; practice note).

d. Speculative decoding under the bound. For the dense worker, verifying $\gamma + 1$ tokens costs one weight read, so the Theorem 3.8 factor applies directly. For the MoE, a batch of n tokens touches, under independent uniform top-k-of-n_e routing,

$$E[\,\text{distinct experts per layer}\,] = n_e(1 - (1 - \tfrac{k}{n_e})^n)$$

For $n_e = 8$, $k = 2$: $n = 1 \to 2$; $2 \to 3.5$; $4 \to 5.47$; $8 \to 7.2$. So verification of $\gamma = 1, 3$ drafts reads 1.75x, 2.7x the expert bytes of a single token — the gain is $\frac{E[\,\text{tokens}\,]}{\text{relative bytes}}$, not $E[\,\text{tokens}\,]$. Mixtral's measured consecutive-expert repetition (20–28% first choice vs 12.5% random, 1.4) lowers these counts somewhat. Measure before enabling speculative decoding on the senior.

e. Prefill is compute-bound, not bandwidth-bound: FLOPs ≈ $2An_{\mathrm{prompt}}$. For the senior on CPU, $2 \cdot 3 \times 10^9 \cdot 8000 \approx 4.8 \times 10^{13}$ FLOP; at ~1 TFLOP/s effective CPU throughput that is ~48 s per 8k-token prompt. Hence prompt/KV caching across turns and the Library's hard budget are first-order for the senior, and the worker (GPU prefill, tens of TFLOP/s) should pre-digest context where possible.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```
        // ---- serving ----
        type QuantType = 'Q4_K_M' | 'Q4_K_S' | 'Q5_K_M' | 'Q6_K' | 'Q8_0' | 'F16' | 'AWQ-INT4-g128' | 'GPTQ-
INT4-g128';
        interface ModelShape { paramsTotal: number; paramsActive: number; layers: number; nHeads: number;
nKvHeads: number; headDim: number; hiddenDim: number; vocab: number; nExperts?: number; topK?: number;
ctxTrain: number; }
        interface QuantSpec { type: QuantType; bpwEff: number; group?: number; calib: { source:
'repo'|'c4'|'pile'; nSeq: number; seqLen: number; hash: string }; method: 'rtn'|'gptq'|'awq'|'imatrix'; }
        interface Placement { weightsDevice: 'gpu'|'cpu'; expertsDevice?: 'cpu'; kvDevice: 'gpu'|'cpu';
kvBits: 16 | 8.5 | 4.5; flashAttention: boolean; }
        interface SpecDecodeConfig { draftModel: string; draftQuant: QuantType; gamma: number; alphaMeasured:
number; cMeasured: number; nTokensMeasured: number; enabled: boolean; }
        interface ServingProfile { role: 'worker'|'senior'; model: string; shape: ModelShape; quant:
QuantSpec; placement: Placement; nCtxMax: number; weightBytes: number; kvBytesPerToken: number; tokPerSecBound:
(nCtx: number) => number; spec?: SpecDecodeConfig; }
        interface Telemetry { tokPerSec: number; promptTokPerSec: number; nCtx: number; alpha?: number;
acceptedPerStep?: number; vramUsed: number; ramUsed: number; }

        // ---- quantization artifacts ----
        interface CalibrationSet { sequences: string[]; tokenizer: string; createdAt: string; sourceGlobs:
string[]; }
        interface GptqLayerResult { layer: string; scales: Float16Array; zeros: Uint8Array; group: number;
dampening: number; errorTrace: number; }
        interface AwqLayerResult { layer: string; alpha: number; scales: Float16Array /* per input channel,
folded */; lossBefore: number; lossAfter: number; }

        // ---- Library index ----
        interface Chunk { id: string; path: string; lang: string; symbol?: string; startLine: number; endLine:
number; text: string; tokens: number; hash: string; indexedAt: string; }
        interface Bm25Index { avgdl: number; k1: number; b: number; df: Record<string, number>; postings:
Record<string, Array<[chunkId: string, tf: number]>>; }
        interface DenseIndex { model: string; dim: number; vectors: Float32Array /* nChunks x dim, L2-
normalized */; ids: string[]; }
        interface ColbertIndex { model: string; dim: number /* m, 128 or less */; dtype: 'f16'; nQuery: number
/* N_q = 32 */; offsets: Uint32Array; tokenEmbeddings: Uint16Array /* concatenated n_d x m */; ids: string[];
punctFiltered: true; }
        interface LibraryManifest { root: string; chunks: number; bm25: string; dense: string; colbert:
string; builtAt: string; fileHashes: Record<string, string>; }

        // ---- retrieval results and packing ----
        interface Candidate { chunkId: string; bm25: number; dense: number; maxsim: number; matchedPositions:
number[] /* argmax_j per query token */; }
        interface PackedContext { budgetTokens: number; usedTokens: number; order: Array<{ chunkId: string;
rank: number; slot: 'first'|'last'|'middle'; trimmedTo?: [number, number] }>; queryBefore: true; queryAfter:
true; }
        interface PositionEval { model: string; k: number; accAtIndex: number[]; closedBook: number; oracle:
number; bestMinusWorst: number; }
```

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Setting	Default	Source
Draft model size	~1/50–1/100 of target so $c \leq 0.05$; same tokenizer/family	Leviathan Table 2/4 (T5-small 77M for 11B, $c < 0.05$)
γ	argmax of Theorem 3.8; start 5 for greedy code, 3 for T=1; range 2–10	Leviathan Sections 3.3, 3.5, Table 2
α measurement	≥10K generated tokens on real traffic; re-measure after each LoRA merge	Leviathan Section 3.1
Enable condition	$\alpha > c$	Leviathan Corollary 3.9
FlashAttention	on for both models; block ~128–256 chosen by kernel	Dao Alg. 1, Fig. 2 middle
Weight quant, worker	4-bit, group 128, activation-aware calibration	GPTQ Section 4 (g128 gap 0.1–0.3 PPL); AWQ Table 4
Weight quant, senior	Q4_K_M-class (4.84 bpw) to fit 32 GB; Q5/Q6 only if RAM allows	derivation 2.2(a); AWQ/GPTQ for accuracy expectations

Setting	Default	Source
GPTQ calibration	128 x 2048 tokens; λ = 1% mean diag; B = 128	Frantar Section 3/4
AWQ calibration	16+ sequences from the developer's own traffic; α grid 20 in [0,1]; g128	Lin Figure 8, Eq. 5
KV cache dtype	f16 default; q8_0 when $n_{ctx}^{\max}$ at f16 is below the needed context	derivation 2.2(b); AWQ/GPTQ show weight 8-bit is lossless (0.0004 PPL at Q8_0 per llama.cpp table) — KV must be tested
MoE routing facts	top-2 of 8, GQA 8 KV heads, head dim 128, 32 layers (reference shape)	Mixtral Table 1
Switch load-loss α, capacity factor	10^{-2}; CF 1.0–1.25 (training-only; not used at inference)	Fedus Sections 2.2, Table 1
Chunk size	~100 words for docs; code by symbol, then ~100–200 tokens	RAG Section 3 (100-word chunks)
First-stage k	50 per retriever (BM25 and dense), union	RAG Figure 3 (RAG-Token peaks at 10 docs; more adds little); ColBERT Table 2 (BM25 R@1000 caps rerank)
Final chunks in context	≤ 8–10; never more than 20	Liu Section 5 (20→50 docs: +1–1.5%); Tables 5–7
Placement order	rank 1 first, rank 2 last, others toward the middle; question before and after	Liu Section 4.2, Tables 5–7
ColBERT	$N_q = 32$, $m = 128$ (or 48/24 to save space), f16 storage, punctuation filtered	Khattab Section 4, Table 4
ColBERT training (if fine-tuning on repo triples)	lr 3×10^{-6}, batch 32, pairwise softmax CE	Khattab Section 4.1
Hard context budget	$n_{ctx}^{\max}$ from 2.2(b) minus generation reserve; evidence ≤ 40% of the window	derivation; Liu (middle < closed-book)

2.5 Evaluation: how to know the module works (metrics, from the papers)

1. **Speculative decoding**: measured α (accepted drafts / proposed drafts) over ≥10K tokens; tokens generated per target call vs $\frac{1-\alpha^{\gamma+1}}{1-\alpha}$; wall-time speedup vs Theorem 3.8 prediction; identical outputs at temperature 0 vs non-speculative decoding (exactness check). Leviathan Section 3.1, Table 2.
2. **Bandwidth bound**: measured decode tok/s at $n_{ctx} \in \{1\text{k}, 8\text{k}, 32\text{k}\}$ vs the 2.2(c) bound; report η_{achieved}; flag < 50% as a placement bug (experts crossing PCIe, KV on the wrong device). Prefill tok/s separately.
3. **Memory**: measured VRAM/RAM at load and at full context vs $W + \text{KV}_{\text{tok}} \cdot n + \text{Buf}$; with FlashAttention the buffer must not grow quadratically in n (Dao Theorem 1).
4. **Quantization quality**: perplexity on a held-out slice of the developer's own code and on WikiText-2 for FP16 vs 4-bit (reference deltas: GPTQ 4-bit g128 within 0.1–0.3 PPL for ≥7B; AWQ INT4 g128 ≤ GPTQ; INT3 g128 Llama-7B 5.47 → 6.24 AWQ). Plus pass@1 on the project's own test suite tasks (the verifier cascade's corpus) FP16 vs quantized: require no significant drop.
5. **Retrieval**: build a synthetic set from the repo — (query = docstring or issue text or a symbol's usage; gold = the defining chunk). Report MRR@10, Recall@{5, 10, 50} for BM25 alone, dense alone, hybrid, hybrid + MaxSim rerank (ColBERT Table 2 shape of comparison; expect rerank to lift MRR substantially while R@50 is set by the first stage). Latency per query.
6. **Position sensitivity** (the Lost in the Middle protocol on our models): gold chunk at index 0, $k/2$, $k-1$ among $k \in \{5, 10, 20\}$ retrieved chunks; report accuracy per index, best–worst gap, closed-book and oracle baselines; key-value retrieval with 75/140/300 pairs. Acceptance criterion: middle-position accuracy must stay above closed-book at the chosen k; otherwise lower k.
7. **End-to-end grounding**: answer/edit correctness with vs without the Library at a fixed budget; fraction of generated identifiers that exist in the repo (a proxy for the RAG "more factual, more specific" claim).

2.6 Known pitfalls and failure modes (from the papers' limitations)

- Speculative decoding needs spare parallel capacity; on the CPU-resident MoE a verification batch inflates expert reads (2.2(d)) and may yield no speedup; α falls ~0.1–0.2 from greedy to T=1; rejected drafts add 1.1–1.6x arithmetic (Leviathan Table 1); the i.i.d.-α model overestimates gains on short generations.

- FlashAttention kernels are per-architecture and per-variant (Dao Section 5): a serving build without the kernel for the GPU silently falls back to $O\left(N^2\right)$ buffers and the VRAM plan breaks.
- MoE memory scales with total parameters (47B, not 13B for Mixtral); expert locality can over-subscribe a few experts; prefill on CPU is compute-bound and slow (2.2(e)).
- Switch-style dropped tokens and load-balancing are training-time behaviours; at inference an unbalanced router simply means some experts are hot — good for caching, irrelevant for correctness.
- GPTQ: no activation quantization, no matmul speedup without INT4 kernels; smaller models lose more; dead-unit layers (OPT-66B) quantize badly; the sequential layer order matters (inputs to layer k must come from quantized layers $< k$).
- AWQ: INT2 fails; mixed-precision salient channels are hard to run fast (use scaling); results lean on perplexity — verify on code tasks.
- Lost in the Middle: middle-placed evidence can be worse than no evidence; 7B models are recency-biased, 13B+ U-shaped; beyond the training context length even robust encoder-decoders degrade; more retrieved documents barely help past ~20.
- RAG: retrieval collapse (same documents for every query) when the retriever is trained without useful signal — one reason we freeze it; hallucination is reduced, not removed; index staleness changes answers (re-index on change).
- ColBERT: token-level storage is large at web scale (286 GiB for 8.8M passages at 128 x 4 bytes) — fine for one repo, but keep f16 and consider $m = 48$; BM25 first-stage recall caps reranking (R@1000 81.4 on MS MARCO), hence the dense leg of the hybrid.

2.7 Open questions this section does not settle

- The actual worker and senior checkpoints and their shapes (L, n_{kv}, d_{head}), which fix $\mathrm{KV}_{\mathrm{tok}}$ and $n_{ctx}^{\max}$; and whether a same-family draft model exists for the worker (if not, acceptance rate must be measured with a cross-family draft or a self-drafting scheme, neither covered here).
- Whether llama.cpp's importance-matrix quantization matches GPTQ/AWQ accuracy at 4-bit on code; none of the papers evaluates GGUF k-quants.
- KV-cache quantization accuracy on code generation (not evaluated by any paper here); only weight quantization has numbers.
- How much the MoE speculative-decoding penalty (2.2(d)) is offset by routing locality in the chosen senior; needs measurement.
- Whether a code-trained ColBERT checkpoint is available; otherwise a cross-encoder reranker or a dense encoder used in MaxSim fashion must be evaluated on the repo-derived retrieval set.
- The right chunking of code (by symbol vs fixed token windows) and the ideal k for a 9B worker — Lost in the Middle gives the protocol, not the number for our model.
- Prompt/KV reuse policy across turns for the senior (prefix caching), which the papers do not cover but 2.2(e) shows is decisive for latency.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2211.17192	Fast Inference from Transformers via Speculative Decoding	2022 (v2 2023)	ICML 2023 (venue not on abs page)	none in text	https://arxiv.org/pdf/2211.17192
2205.14135	FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness	2022	NeurIPS 2022 (venue not on abs page)	https://github.com/HazyResearch/flash-attention	https://arxiv.org/pdf/2205.14135

arXiv id	Title	Year	Venue	Code	PDF
2101.03961	Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity	2021 (v3 2022)	JMLR 2022 (venue not on abs page)	none in fetched text (Mesh-TF pseudocode in App. F)	https://arxiv.org/pdf/2101.03961
2401.04088	Mixtral of Experts	2024	arXiv preprint	https://github.com/mistralai/mistral-src	https://arxiv.org/pdf/2401.04088
2210.17323	GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers	2022 (v2 2023)	ICLR 2023	https://github.com/IST-DASLab/gptq	https://arxiv.org/pdf/2210.17323
2306.00978	AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration	2023 (v6 2026)	MLSys 2024 (Best Paper)	https://github.com/mit-han-lab/llm-awq	https://arxiv.org/pdf/2306.00978
2307.03172	Lost in the Middle: How Language Models Use Long Contexts	2023	TACL 2023	project page nelsonliu.me/papers/lost-in-the-middle	https://arxiv.org/pdf/2307.03172
2005.11401	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	2020 (v4 2021)	NeurIPS 2020 (venue not on abs page)	https://github.com/huggingface/transformers (examples/rag)	https://arxiv.org/pdf/2005.11401
2004.12832	ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT	2020	SIGIR 2020 (venue not on abs page)	github.com/stanford-futuredata/ColBERT (not in fetched text)	https://arxiv.org/pdf/2004.12832